

Практическая работа 13. Создание интерактивного приложение Dice Roller.

В этом уроке вы создадите интерактивное приложение **Dice Roller**, которое позволит пользователям нажимать на кнопку, чтобы бросить кости. Результат броска отображается на экране в виде изображения.

Вы используете Jetpack Compose с Kotlin для создания макета приложения, а затем пишете бизнес-логику для обработки того, что происходит при нажатии на композитную кнопку.

Необходимые условия

- Умение создать и запустить базовое приложение Compose в Android Studio.
- Знакомство с тем, как использовать компонуемый текст в приложении.
- Знание того, как извлечь текст в строковый ресурс, чтобы упростить перевод приложения и повторное использование строк.
- Знание основ программирования на языке Kotlin.

Что вы узнаете

- Как добавить композитную кнопку в приложение для Android с помощью Compose.
- Как добавить поведение к кнопке, создаваемой в приложении для Android с помощью Compose.
- Как открывать и изменять код Activity приложения для Android.

Что вы создадите

- Интерактивное приложение для Android под названием **Dice Roller**, которое позволяет пользователям бросать кости и показывает им результат броска.

Вот как будет выглядеть приложение после завершения этого коделаба:

Установите базовую линию

- Создайте проект
- В Android Studio нажмите Файл > Новый > Новый проект.
- В диалоговом окне «Новый проект» выберите Empty Activity и нажмите Next.

В поле Имя введите **Dice Roller**. В поле Minimum SDK выберите в меню минимальный уровень API 33 (Tiramisu), а затем нажмите Finish.

Создание инфраструктуры макета

Предварительный просмотр проекта Чтобы просмотреть проект, выполните следующие действия:

- Нажмите Build & Refresh на панели Split или Design.
- Теперь вы должны увидеть предварительный просмотр в панели Design. Если он выглядит маленьким, не волнуйтесь, потому что он меняется при изменении макета.

Перестройте код примера

Вам нужно изменить часть сгенерированного кода, чтобы он больше походил на тему приложения для броска костей.

Как вы видели на скриншоте финального приложения, там есть изображение игральной кости и кнопка для ее бросания. Вам предстоит структурировать составные функции, чтобы отразить эту архитектуру.

Чтобы реструктурировать код примера:

- Удалите функцию `GreetingPreview()`.
- Создайте функцию `DiceWithButtonAndImage()` с аннотацией `@Composable`. Эта композитная функция представляет компоненты пользовательского интерфейса макета, а также содержит логику нажатия на кнопку и отображения изображения.
- Удалите функцию `Greeting(name: String, modifier: Modifier = Modifier)`.
- Создайте функцию `DiceRollerApp()` с аннотациями `@Preview` и `@Composable`. Поскольку это приложение состоит только из кнопки и изображения, считайте эту составную функцию самым приложением. Поэтому она и называется функцией `DiceRollerApp()`.

MainActivity.kt

```
@Preview
@Composable
fun DiceRollerApp() {

}

@Composable
fun DiceWithButtonAndImage() {

}
```

Поскольку вы удалили функцию `Greeting()`, вызов `Greeting(«Android»)` в теле лямбды `DiceRollerTheme()` выделен красным цветом. Это потому, что компилятор больше не может найти ссылку на эту функцию.

- Удалите весь код внутри лямбды `setContent{}`, находящейся в методе `onCreate()`. В теле лямбды `setContent{}` вызовите лямбду `DiceRollerTheme{}`, а затем внутри лямбды `DiceRollerTheme{}` вызовите функцию `DiceRollerApp()`.

MainActivity.kt

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContent {  
        DiceRollerTheme {  
            DiceRollerApp()  
        }  
    }  
}
```

- В функции DiceRollerApp() вызовите функцию DiceWithButtonAndImage(). MainActivity.kt

```
@Preview  
@Composable  
fun DiceRollerApp() {  
    DiceWithButtonAndImage()  
}
```

Добавить модификатор

Compose использует объект **Modifier**, который представляет собой коллекцию элементов, украшающих или изменяющих поведение элементов пользовательского интерфейса Compose. Вы используете его для стилизации компонентов пользовательского интерфейса приложения Dice Roller.

Чтобы добавить модификатор:

- Измените функцию DiceWithButtonAndImage(), чтобы она принимала аргумент модификатора типа Modifier и присваивала ему значение Modifier по умолчанию. MainActivity.kt

```
@Composable  
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {  
}
```

Предыдущий фрагмент кода может сбить вас с толку, поэтому давайте разберем его подробнее. Функция позволяет передавать параметр-модификатор. Значением по умолчанию параметра-модификатора является объект **Modifier**, отсюда и фрагмент = Modifier в сигнатуре метода. Значение параметра по умолчанию позволяет всем, кто вызовет этот метод в будущем, решить, передавать ли значение параметра. Если они передадут свой собственный объект Modifier, то смогут настроить поведение и оформление пользовательского интерфейса. Если они решат не передавать объект Modifier, то параметр примет значение по умолчанию, то есть обычный объект Modifier. Эту практику можно применить к любому параметру.

Примечание: Утверждение `import androidx.compose.ui.Modifier` импортирует пакет `androidx.compose.ui.Modifier`, который позволяет ссылаться на объект `Modifier`.

Теперь, когда у композита `DiceWithButtonAndImage()` появился параметр `modifier`, передавайте модификатор при вызове композита. Поскольку сигнатура метода функции `DiceWithButtonAndImage()` изменилась, при ее вызове следует передать объект `Modifier` с нужными украшениями. Класс `Modifier` отвечает за украшение или добавление поведения к композиту в функции `DiceRollerApp()`. В данном случае необходимо добавить несколько важных украшений в объект `Modifier`, который передается в функцию `DiceWithButtonAndImage()`. Вы можете задаться вопросом, зачем вообще передавать аргумент `Modifier`, если он есть по умолчанию. Причина в том, что составные объекты могут подвергаться рекомпозиции, что, по сути, означает повторное выполнение блока кода в методе `@Composable`. Если объект `Modifier` создается в блоке кода, он потенциально может быть создан заново, а это неэффективно. **Рекомпозиция** будет рассмотрена позже в этом уроке.

MainActivity.kt

```
DiceWithButtonAndImage(modifier = Modifier)
```

- Привяжите метод `fillMaxSize()` к объекту `Modifier`, чтобы макет заполнил весь экран. Этот метод указывает, что компоненты должны заполнить все доступное пространство. Ранее в этом руководстве вы видели скриншот конечного пользовательского интерфейса приложения `Dice Roller`. Примечательной особенностью является то, что игральные кости и кнопка расположены по центру экрана. Метод `wrapContentSize()` указывает, что доступное пространство должно быть по крайней мере таким же большим, как и компоненты внутри него. Однако, поскольку используется метод `fillMaxSize()`, если компоненты внутри макета меньше доступного пространства, в метод `wrapContentSize()` можно передать объект `Alignment`, который укажет, как компоненты должны выравниваться в доступном пространстве.

MainActivity.kt

```
DiceWithButtonAndImage(modifier = Modifier  
    .fillMaxSize()  
)
```

Примечание: Операторы импорта для `fillMaxSize()` и `wrapContentSize()` - `import androidx.compose.foundation.layout.fillMaxSize` и `import androidx.compose.foundation.layout.wrapContentSize`, соответственно.

- Цепляем метод `wrapContentSize()` к объекту `Modifier`, а затем передаем `Alignment.Center` в качестве аргумента для центрирования компонентов. `Alignment.Center` определяет, что компонент центрируется как по вертикали, так и по горизонтали. MainActivity.kt

```
DiceWithButtonAndImage(modifier = Modifier  
    .fillMaxSize()  
    .wrapContentSize(Alignment.Center)  
)
```

Примечание: Оператор импорта для объекта Alignment имеет вид `import androidx.compose.ui.Alignment`.

Создание вертикального макета

В Compose вертикальные макеты создаются с помощью функции `Column()`.

Функция `Column()` - это композитный макет, который размещает свои дочерние элементы в вертикальной последовательности. В предполагаемом дизайне приложения вы можете видеть, что изображение игральной кости отображается вертикально над кнопкой броска:

Чтобы создать вертикальный макет:

- В функцию `DiceWithButtonAndImage()` добавьте функцию `Column()`.

Примечание: Оператор импорта для композита `Column` - `import androidx.compose.foundation.layout.Column`.

Передайте аргумент модификатора из сигнатуры метода `DiceWithButtonAndImage()` в аргумент модификатора функции `Column()`. Аргумент модификатора гарантирует, что композиты в функции `Column()` будут придерживаться ограничений, вызванных для экземпляра модификатора.

- Передайте аргумент `horizontalAlignment` функции `Column()`, а затем установите для него значение `Alignment.CenterHorizontally`. Это гарантирует, что дочерние элементы в колонке будут отцентрированы на экране устройства относительно ширины.

MainActivity.kt

```
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {  
    Column (  
        modifier = modifier,  
        horizontalAlignment = Alignment.CenterHorizontally  
    ) {}  
}
```

Добавьте кнопку

В файле `strings.xml` добавьте строку и установите для нее значение `Roll`. `res/values/strings.xml`

```
<string name="roll">Roll</string>
```

В тело лямбды `Column()` добавьте функцию `Button()`.

Примечание: Оператором импорта для композитной кнопки является `import androidx.compose.material3.Button`.

В файле MainActivity.kt добавьте функцию Text() к Button() в теле лямбда-функции. Передайте идентификатор строкового ресурса строки ролика в функцию stringResource() и передайте результат функции Text composable.

MainActivity.kt

```
Column(  
    modifier = modifier,  
    horizontalAlignment = Alignment.CenterHorizontally  
) {  
    Button(onClick = { /*TODO*/ }) {  
        Text(stringResource(R.string.roll))  
    }  
}
```

Примечание: Если вы автозаполните функцию `Button()`, появится аргумент `onClick = { /TODO/ }`. Если вы не заполняете его автоматически или Android Studio не позволяет вам это сделать, вы можете реализовать этот аргумент самостоятельно в качестве заполнителя.

Примечание: Оператор импорта для функции `stringResource` имеет вид `import androidx.compose.ui.res.stringResource`.

Добавьте изображение

Еще один важный компонент приложения - изображение игровой кости, которое показывает результат, когда пользователь нажимает кнопку Roll. Вы добавляете изображение с помощью Image composable, но для этого требуется ресурс изображения, поэтому сначала вам нужно загрузить несколько изображений, предусмотренных для этого приложения.

Загрузите изображения игровых костей

- Откройте этот URL-адрес, чтобы загрузить zip-файл с изображениями игровых костей на свой компьютер, а затем дождитесь окончания загрузки.
- Найдите файл на своем компьютере. Скорее всего, он находится в папке «Загрузки».
- Распакуйте zip-файл, чтобы создать новую папку `dice_images`, содержащую шесть файлов с изображениями игровых костей со значениями от 1 до 6.
- Добавьте изображения игровых костей в свое приложение В Android Studio нажмите View > Tool Windows > Resource Manager. Нажмите + > Import Drawables, чтобы открыть браузер файлов.
- Найдите и выберите папку с изображениями шести игровых костей и загрузите их. Загруженные изображения будут выглядеть следующим образом.
- Нажмите кнопку Далее.

Появится диалоговое окно Import Drawables, в котором показано, где в файловой структуре находятся файлы ресурсов.

- Нажмите кнопку Импорт, чтобы подтвердить, что вы хотите импортировать шесть изображений. Изображения должны появиться на панели Менеджера ресурсов.

Важно! Вы можете ссылаться на эти изображения в коде Kotlin с помощью их идентификаторов ресурсов: R.drawable.dice_1 R.drawable.dice_2 R.drawable.dice_3 R.drawable.dice_4 R.drawable.dice_5 R.drawable.dice_6

Отличная работа! В следующем задании вы используете эти изображения в своем приложении.

Добавьте композитное изображение

Изображение игральной кости должно появиться над кнопкой Roll. Compose по своей природе размещает компоненты пользовательского интерфейса последовательно. Другими словами, тот компонент, который объявлен первым, отображается первым. Это может означать, что первое объявление отображается выше или раньше композита, объявленного после него. Компоненты, находящиеся внутри композита Column, будут отображаться на устройстве выше/ниже друг друга. В этом приложении вы используете столбец для вертикальной укладки композитов, поэтому тот композит, который объявлен первым в функции Column(), отображается перед композитом, объявленным после него в той же функции Column().

Чтобы добавить композит Image:

- В теле функции Column() создайте функцию Image() перед функцией Button(). MainActivity.kt

```
Column(  
    modifier = modifier,  
    horizontalAlignment = Alignment.CenterHorizontally  
) {  
    Image()  
    Button(onClick = { /*TODO*/ }) {  
        Text(stringResource(R.string.roll))  
    }  
}
```

Примечание: Оператор импорта для композита Image должен быть import androidx.compose.foundation.Image.

- Передайте функции Image() аргумент painter, а затем присвойте ей значение painterResource, которое принимает аргумент id ресурса drawable. На данный момент передайте следующий идентификатор ресурса: R.drawable.dice_1.

MainActivity.kt

```
Image(  
    painter = painterResource(R.drawable.dice_1)  
)
```

Примечание: Оператор импорта для функции `painterResource` - `import androidx.compose.ui.res.painterResource`.

Примечание: Позже вы измените значение, передаваемое для идентификатора ресурса. Пока же его следует передавать по умолчанию, чтобы код компилировался для целей предварительного просмотра.

Каждый раз, когда вы создаете изображение в своем приложении, вы должны предоставить так называемое «описание содержимого». Описание содержимого - важная часть разработки Android. Они прикрепляют описания к соответствующим компонентам пользовательского интерфейса, чтобы повысить их доступность. Вы можете передать описание содержимого изображению в качестве параметра. `MainActivity.kt`

```
Image(  
    painter = painterResource(R.drawable.dice_1),  
    contentDescription = "1"  
)
```

Примечание: Приведенное выше описание содержимого пока что является условным. Оно будет обновлено в одном из последующих разделов этого урока.

Теперь все необходимые компоненты пользовательского интерфейса присутствуют. Но кнопка и изображение немного теснят друг друга.

- Чтобы исправить это, добавьте композит `Spacer` между композитами `Image` и `Button`. `Spacer` принимает модификатор в качестве параметра. В данном случае изображение находится над кнопкой, поэтому между ними должно быть вертикальное пространство. Поэтому высота модификатора может быть установлена для применения к спейсеру. Попробуйте установить высоту `16.dp`. Обычно размеры `dp` изменяются с шагом `4.dp`.

`MainActivity.kt`

```
Spacer(modifier = Modifier.height(16.dp))
```

Примечание: Импорт для композита `Spacer`, модификатора высоты и `dp` следующий:

```
import androidx.compose.foundation.layout.height  
import androidx.compose.foundation.layout.Spacer  
import androidx.compose.ui.unit.dp
```


- В панели предварительного просмотра нажмите Build & Refresh. Вы должны увидеть нечто похожее на это изображение:

Постройте логику броска кубиков

Теперь, когда все необходимые компоненты присутствуют, вы модифицируете приложение так, чтобы нажатие на кнопку бросало кости.

- Сделайте кнопку интерактивной

В функции `DiceWithButtonAndImage()` перед функцией `Column()` создайте переменную `result` и установите ее в значение 1.

- Посмотрите на составную кнопку `Button`. Вы заметите, что ей передается параметр `onClick`, который установлен в пару фигурных скобок с комментарием `/TODO/` внутри скобок. В данном случае скобки представляют собой так называемую лямбду, а область внутри скобок - тело лямбды.

Когда функция передается в качестве аргумента, ее также можно назвать «обратным вызовом callback».

```
Button(onClick = { /*TODO*/ })
```

Лямбда - это функция-литерал, которая является функцией, как и любая другая, но вместо того, чтобы быть объявленной отдельно с ключевым словом `fun`, она записывается в строку и передается как выражение. Композит `Button` ожидает, что в качестве параметра `onClick` будет передана функция. Это идеальное место для использования лямбды, и в этом разделе вы будете писать тело лямбды.

-В функции `Button()` удалите комментарий `/TODO/` из значения тела лямбды параметра `onClick`. Бросок игровой кости является случайным. Чтобы отразить это в коде, необходимо использовать правильный синтаксис для генерации случайного числа. В Kotlin вы можете использовать метод `random()` для диапазона чисел. В теле лямбды `onClick` установите переменную `result` в диапазон от 1 до 6, а затем вызовите метод `random()` на этом диапазоне. Помните, что в Kotlin диапазоны обозначаются двумя точками между первым и последним числом в диапазоне.

```
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    var result = 1
    Column(
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Image(
            painter = painterResource(R.drawable.dice_1),
            contentDescription = "1"
        )
    }
}
```

```
        Spacer(modifier = Modifier.height(16.dp))
        Button(onClick = { result = (1..6).random() }) {
            Text(stringResource(R.string.roll))
        }
    }
}
```

Теперь кнопка доступна для нажатия, но ее нажатие пока не вызовет никаких визуальных изменений, потому что вам еще нужно создать эту функциональность.

- Добавьте условие в приложение для бросания костей В предыдущем разделе вы создали переменную `result` и жестко закодировали ее в значение 1. В конечном итоге значение переменной `result` сбрасывается при нажатии на кнопку Roll, и это должно определять, какое изображение будет показано.

По умолчанию Composables являются **stateless**, что означает, что они не имеют значения и могут быть перекомпонованы системой в любой момент, что приведет к сбросу значения. Однако Compose предоставляет удобный способ избежать этого. Композитные функции могут хранить объект в памяти с помощью функции **remember** composable.

- Сделайте переменную `result` запоминаемой композитной.
- Remember composable требует передачи функции.
- В теле remember composable передайте функцию **mutableStateOf()**, а затем передайте ей 1 аргумент.

Функция **mutableStateOf()** возвращает наблюдаемую переменную. Подробнее о наблюдаемых вы узнаете позже, а пока это означает, что при изменении значения переменной `result` запускается перекомпозиция, отражается значение `result` и обновляется пользовательский интерфейс.

```
var result by remember { mutableStateOf(1) }
```

Примечание: Операторы `import androidx.compose.runtime.mutableStateOf` и `import androidx.compose.runtime.remember` импортируют пакеты, необходимые для функции `mutableStateOf()` и композита `remember`.

Следующие операторы импорта также необходимы для импорта необходимых функций расширения `State`:

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
```

Теперь, когда кнопка нажимается, переменная `result` обновляется значением случайного числа.

Теперь переменную `result` можно использовать для определения того, какое изображение нужно показать.

- Под инстанцией переменной `result` создайте неизменяемую переменную `imageResource`, установленную на выражение `when`, которое принимает переменную `result`, а затем установите каждый возможный результат на свой `drawable`.

```
val imageResource = when (result) {  
    1 -> R.drawable.dice_1  
    2 -> R.drawable.dice_2  
    3 -> R.drawable.dice_3  
    4 -> R.drawable.dice_4  
    5 -> R.drawable.dice_5  
    else -> R.drawable.dice_6  
}
```

- Измените идентификатор, передаваемый в параметр `painterResource` композита `Image`, с `drawable R.drawable.dice_1` на переменную `imageResource`.
- Измените параметр `contentDescription` композитного `Image`, чтобы он отражал значение переменной `result`, преобразовав переменную `result` в строку с помощью функции `toString()` и передав ее в качестве `contentDescription`.

```
Image(  
    painter = painterResource(imageResource),  
    contentDescription = result.toString()  
)
```

- Запустите ваше приложение. Теперь ваше приложение **Dice Roller** должно полностью работать!

Заключение

Вы создали интерактивное приложение **Dice Roller** для Android с помощью Compose!

Резюме

- Определение композитных функций.
- Создание макетов с помощью композиций.
- Создайте кнопку с помощью композитной функции `Button`.
- Импорт ресурсов для рисования.
- Отображение изображения с помощью композиции `Image`.
- Создание интерактивного пользовательского интерфейса с помощью композитов.
- Используйте компонент `remember` для сохранения объектов в композиции в памяти.
- Обновите пользовательский интерфейс с помощью функции `mutableStateOf()` для создания - наблюдаемого объекта.

